

UNIVERSIDADE FEDERAL DE SÃO PAULO
Instituto de Ciência e Tecnologia
Álgebra Linear Computacional

Eigenfaces: Reconhecimento Facial via Decomposição de Subespaços e PCA

Marcos Paulo Crisp Modesto – RA: 178.190
Guilherme da Rocha Verneck Braz – RA: 176.497
Luis Inácio de Campos Alves – RA: 178.189

Professor: Dr. LUIZ LEDUINO SALLES NETO

São José dos Campos
2026

Resumo

Este projeto tem como objetivo aplicar conceitos de Álgebra Linear Computacional no problema de reconhecimento de faces. Para lidar com a alta complexidade imposta por imagens de alta resolução, utilizou da técnica de Análise de Componentes Principais (PCA), que permite reduzir a dimensionalidade dos dados e identificar padrões relevantes em imagens. A metodologia consiste em transformar as imagens capturadas em vetores e aplicar decomposições matriciais para calcular autovalores e autovetores, possibilitando a construção das chamadas *eigenfaces*. Todo o processo lógico e matemático teve sua implementação feita em Python. O principal resultado obtido é um protótipo funcional de controle de acesso facial inspirado em aplicações reais, como portarias de prédios, que compara e reconhece diferentes imagens com base em suas similaridades geométricas em um espaço vetorial de menor dimensão.

Abstract

This project aims to apply Computational Linear Algebra concepts to the face recognition problem. To deal with the high complexity imposed by high-resolution images, the Principal Component Analysis (PCA) technique was used, which allows reducing data dimensionality and identifying relevant patterns in images. The methodology consists of transforming captured images into vectors and applying matrix decompositions to compute eigenvalues and eigenvectors, enabling the construction of the so-called *eigenfaces*. The entire logical and mathematical process was implemented in Python. The main result obtained is a functional prototype of a biometric access control system inspired by real-world applications, such as building entrance systems, which compares and recognizes different images based on their geometric similarities in a lower-dimensional vector space.

Lista de Figuras

1 Limiar de aceitação de 30.000, falso positivo. 6
2 Limiar de aceitação de 300, falso negativo. 6
3 Exemplos da imagem 112x92 vetorizada e reconstruída. 7
4 Autovetores gerados a partir de uma pessoa no banco de dados, utilizando óculos. 8
5 Autovetores gerados de uma pessoa no banco de dados, capturada com e sem óculos. 8
6 Representação das Eigenfaces para um banco de dados contendo duas pessoas distintas. 8
7 Representação das Eigenfaces para um banco de dados contendo quatro pessoas distintas. 9
8 Gráfico de Acurácia vs. Número de Componentes Principais (k). 9
9 k=3 algoritmo perde eficiência 10
10 k=60 algoritmo eficiênte 10
11 Oscilação na predição de classe devido à sensibilidade a pequenas variações de pose. 11

Sumário

1 Introdução **1**

2 Fundamentação Teórica **1**

3 Metodologia e Implementação Computacional **3**
3.1 Aquisição e Pré-Processamento de Dados 3
3.2 Separação do Conjunto de Treino e Teste 3
3.3 Projeção e Classificação Geométrica 4
3.4 Ferramentas e Bibliotecas 5
3.5 Análise de Complexidade e Estabilidade 5

4 Resultados e Discussão **5**
4.1 Apresentação de Dados e Desempenho Computacional 5
4.2 Aplicações Visuais 7
4.3 Acurácia em componentes principais (k) 9
4.4 Sensibilidade a Acessórios e Variações de Pose 11
4.5 Interpretação Analítica 12

5 Conclusão **12**

Apêndice A – Implementação da Rotina Matemática (PCA) **13**

1 Introdução

O reconhecimento facial se tornou uma tecnologia muito útil quando olhamos para a segurança e o controle de acesso no mundo moderno, presente de forma cotidiana no acesso a celulares, aplicativos importantes, portarias eletrônicas e sistemas de vigilância. Do ponto de vista computacional, comparar faces humanas de forma automatizada é um desafio de alto custo numérico, as imagens digitais são compostas por milhares de pixels que atuam como variáveis independentes, gerando uma sobrecarga de dados, visto que, uma única imagem de 92×112 pixels, por exemplo, já corresponde a mais de 10 mil dimensões.

A Álgebra Linear apresenta as bases computacionais adequadas para a solução desse problema. Ao interpretar a imagem bidimensional não apenas como um amontuado de pixels, mas sim como matrizes e vetores, torna-se possível a extração de dados significativos. O uso da *Principal Component Analysis* (PCA), ou Análise de Componentes Principais, se justifica pela sua eficiência em isolar apenas as variâncias vitais dos rostos, removendo o ruído da imagem e transferindo o processamento para um novo subespaço de baixa dimensão.

Essa redução é possível porque, embora cada rosto ocupe teoricamente um espaço de milhares de dimensões, as variações reais entre diferentes faces, iluminação, expressão, geometria, tendem a se concentrar em um subespaço muito menor. A PCA identifica justamente as direções (autovetores) ao longo das quais essa variância é maior, permitindo descartar informações redundantes sem perda significativa de capacidade discriminativa.

O objetivo geral do trabalho é aplicar técnicas de Álgebra Linear Computacional no desenvolvimento de um modelo de reconhecimento facial. Dentre os objetivos específicos, incluem-se o estudo e a implementação dos conceitos de PCA (autovalores e autovetores) no algoritmo, finalizando com a criação de um protótipo prático de controle de acesso, cujos resultados e desempenho serão avaliados e interpretados.

2 Fundamentação Teórica

Para fundamentar a implementação do método de *Eigenfaces*, diversos conceitos de Álgebra Linear Computacional são articulados para promover a transição do espaço de imagens para o subespaço de características:

- **Vetorização de Imagens:** o mapeamento de uma imagem bidimensional, com resolução padronizada de 92×112 pixels, para um vetor unidimensional pertencente ao espaço vetorial $\mathbb{R}^d$, onde a dimensão resultante é $d = 92 \times 112 = 10304$.

- **Face Média e Centralização:** dado um conjunto de treinamento com n imagens vetorizadas $\vec{x}_1, \ldots, \vec{x}_n \in \mathbb{R}^d$, é calculada a face média do banco de dados:

$$\bar{\vec{x}} = \frac{1}{n} \sum_{i=1}^{n} \vec{x}_i, \tag{1}$$

  onde cada imagem é então centralizada através da subtração $\vec{\phi}_i = \vec{x}_i - \bar{\vec{x}}$. Definindo a matriz de dados $X \in \mathbb{R}^{n \times d}$, cujas linhas correspondem aos vetores de desvio $\vec{\phi}_i^T$, a matriz de dispersão (*scatter matrix*) C, proporcional à covariância amostral, é dada por:

$$C = X^T X \in \mathbb{R}^{d \times d}. \tag{2}$$

- **O Problema Espectral:** o núcleo matemático da técnica PCA consiste em solucionar a clássica equação de autovalores e autovetores:

$$C\vec{v} = \lambda\vec{v}. \tag{3}$$

Nesta formulação, os autovalores ($\lambda \geq 0$) quantificam a variância capturada na direção correspondente, e os autovetores ($\vec{v}$) estabelecem as direções de máxima variância, formando visualmente as *eigenfaces*. Como C é uma matriz real, simétrica e positiva semidefinida, o Teorema Espectral garante autovalores reais e não-negativos, além de assegurar que autovetores associados a autovalores distintos sejam ortogonais entre si, viabilizando a construção de uma base ortonormal para o subespaço.

- **O Truque do *Snapshot* (Turk & Pentland, 1991):** a matriz C possui dimensões proibitivas (10304×10304), o que torna sua diagonalização direta um problema de altíssimo custo computacional Turk and Pentland (1991). Contorna-se este obstáculo calculando os autovetores da matriz auxiliar $C' = XX^T$, de dimensão $n \times n$, onde $n \ll d$. Se $\vec{u}_i$ é autovetor de C' associado ao autovalor λ_i, então $\vec{v}_i = X^T\vec{u}_i$ é autovetor de C associado ao mesmo autovalor, o que se verifica diretamente:

$$C\vec{v}_i = (X^TX)(X^T\vec{u}_i) = X^T(XX^T)\vec{u}_i = X^T(\lambda_i\vec{u}_i) = \lambda_i(X^T\vec{u}_i) = \lambda_i\vec{v}_i. \tag{4}$$

Sob a ótica estrutural da Álgebra Linear Computacional, essa redução dimensional encontra respaldo direto na **Decomposição em Valores Singulares (SVD)**, que atesta que as matrizes X^TX e XX^T compartilham rigorosamente os mesmos autovalores não nulos.

- **Mudança de Base e Projeção Ortogonal:** selecionados os k autovetores dominantes $\vec{v}_1, \ldots, \vec{v}_k$ (normalizados pela norma euclidiana), qualquer rosto centralizado $\vec{\phi}$ é projetado ortogonalmente no subespaço dimensionalmente reduzido, gerando um vetor de pesos de projeção $\vec{w} \in \mathbb{R}^k$:

$$w_j = \vec{v}_j^T\vec{\phi}, \quad j = 1, \ldots, k. \tag{5}$$

O reconhecimento facial converte-se, assim, em um problema geométrico clássico: minimizar a distância euclidiana entre o vetor de pesos $\vec{w}$ da imagem de entrada e os vetores de pesos dos indivíduos armazenados no banco de dados.

- **Erro de Reconstrução e Variância Explicada:** truncar a base para $k < n$ componentes introduz um erro de reconstrução natural, porém concentra a informação vital para a classificação. A proporção da variância total retida ao selecionar os k maiores componentes é calculada por:

$$\text{Variância Explicada}(k) = \frac{\sum_{i=1}^{k}\lambda_i}{\sum_{i=1}^{n}\lambda_i}. \tag{6}$$

Esta métrica atua como o principal critério analítico para a determinação empírica do valor ótimo de k, garantindo o limiar ideal entre compressão de dados e acurácia no reconhecimento.

3 Metodologia e Implementação Computacional

O algoritmo de reconhecimento facial foi inteiramente desenvolvido em linguagem **Python**, sem utiliazar bibliotecas prontas de classificação biométrica em favor da implementação manual do instrumental de Álgebra Linear utilizando a biblioteca **NumPy**. A interface de captura de imagens foi gerenciada pela biblioteca **OpenCV**. A arquitetura do sistema foi modularizada em dois scripts independentes: o primeiro (**capturar_rostos.py**) atua exclusivamente na construção e padronização do banco de dados de forma automatizada; o segundo (**EigenFacePCA.py**) é responsável por todo o núcleo matemático, cálculo do espectro matricial e execução da interface de controle de acesso em tempo real. A metodologia computacional foi dividida em quatro etapas principais.

3.1 Aquisição e Pré-Processamento de Dados

A captura das faces foi realizada em tempo real via webcam. Utilizou-se o algoritmo de detecção facial *Haar Cascade* (através do arquivo pré-treinado `haarcascade_frontalface_default.xml`) para localizar e isolar a região de interesse. Por exigência estrutural da formulação algébrica, todas as imagens capturadas foram convertidas para escala de cinza e redimensionadas para uma resolução padronizada de 92×112 pixels. Em seguida, cada imagem foi vetorizada para um vetor linha unidimensional, garantindo que o espaço de características operasse invariavelmente em $\mathbb{R}^{10304}$.

Para cada indivíduo cadastrado, foram capturadas 30 a 80 fotografias, intercaladas por um pequeno intervalo de tempo entre capturas consecutivas. Um atraso de 100 milissegundos (`cv2.waitKey(100)`) foi intencionalmente adicionado entre as capturas para garantir que o operador tivesse tempo de realizar leves movimentos, capturando assim as variações angulares necessárias para a construção das Eigenfaces.

3.2 Separação do Conjunto de Treino e Teste

Para reduzir o vazamento de dados (*data leakage*) entre indivíduos distintos, o banco de imagens foi dividido utilizando a variável `proporcao_teste = 0.2` (80% para treinamento e 20% para teste), com separação estratificada por indivíduo (*split* por pessoa). Essa estratégia assegura que nenhuma imagem da mesma pessoa presente no conjunto de teste tenha sido também usada no treinamento do modelo. Porém, todas as fotos de um mesmo indivíduo foram capturadas em uma única sessão contínua (mesma iluminação e pose semelhante), de modo que ainda existe alguma correlação temporal entre os conjuntos de treino e teste de uma mesma pessoa, limitação discutida na Seção.

3.3 Projeção e Classificação Geométrica

A classificação de uma nova face ocorre pela sua projeção ortogonal no subespaço de dimensão reduzida k, gerado pelas k melhores *Eigenfaces*. O modelo calcula os pesos da imagem de teste através do produto interno e aplica o algoritmo de classificação por **Vizinho Mais Próximo** (Nearest Neighbor), calculando a distância euclidiana entre o vetor de pesos da face de teste e os vetores do banco de dados conhecido.

A escolha do Vizinho Mais Próximo como classificador se justifica pela sua simplicidade geométrica direta. Uma vez que o PCA já reorganiza o espaço de características de modo a concentrar a variância nos primeiros componentes, comparar distâncias euclidianas nesse subespaço reduzido é suficiente para capturar a similaridade entre faces, sem exigir o treinamento de um classificador adicional sobre os pesos projetados.

A fim de lidar com rostos não cadastrados, estabeleceu-se empiricamente o parâmetro **LIMIAR_ACEITACAO = 3000.0**. Se a distância euclidiana mínima calculada for superior a este limiar, o sistema infere que a projeção se encontra distante do *Eigenfaces* esperado, rejeitando a imagem e rotulando o indivíduo como "Desconhecido". A calibração é aumentada ou diminuida mediante a testes, valores muito baixos tornam o sistema restritivo; valores muito altos comprometem controle de acesso.

O fluxo de dados da implementação é sintetizado no Algoritmo 1.

Algorithm 1 Reconhecimento Facial via Eigenfaces (NumPy)

procedure TreinarEigenfaces($X \in \mathbb{R}^{n \times d}$)
  $\vec{\bar{x}} \leftarrow$ média das linhas de X ▷ Face média
  $X_c \leftarrow X - \vec{\bar{x}}$ ▷ Centralização
  $C' \leftarrow X_c X_c^T$ ▷ Truque dimensional, $O(n^3)$
  $\lambda, U \leftarrow$ autovalores e autovetores de C', ordenados decrescentemente
  $V \leftarrow X_c^T U$, normalizado por colunas ▷ Eigenfaces
  return $\vec{\bar{x}}, V, \lambda$
end procedure
procedure Classificar($\vec{\Gamma}, \vec{\bar{x}}, V_k$, pesos de treino, θ)
  $\vec{w} \leftarrow (\vec{\Gamma} - \vec{\bar{x}})^T V_k$ ▷ Projeção no subespaço
  $\epsilon_i \leftarrow \|\vec{w} - \text{pesos de treino}^{(i)}\|_2 \ \forall i$
  if $\min(\epsilon) < \theta$ **then**
    return identidade associada a $\arg\min(\epsilon)$
  else
    return *"Desconhecido"*
  end if
end procedure

3.4 Ferramentas e Bibliotecas

A implementação de todas as estruturas e equações lógicas mencionadas foi realizada através da linguagem **Python**. Para a computação vetorial utilizou-se a biblioteca **NumPy**. Processamento de imagem em tempo real, assim como recortes automáticos das faces baseados em classificadores, utilizaram a **OpenCV**. Por fim, métodos visuais estatísticos ficaram a encargo da biblioteca **Matplotlib**.

3.5 Análise de Complexidade e Estabilidade

Um dos principais desafios computacionais da Análise de Componentes Principais (PCA) está na dimensão da matriz de covariância $C \in \mathbb{R}^{d \times d}$, em que d representa o número de pixels da imagem vetorizada. A diagonalização direta dessa matriz possui custo computacional da ordem de $O(d^3)$, tornando-se inviável quando d é muito grande. Neste trabalho, por exemplo, cada imagem possui 10304 pixels, o que resultaria em uma matriz de covariância de grandes dimensões e elevado custo de processamento e memória.

Para reduzir esse custo, foi utilizada a técnica apresentada na Seção 2, que consiste em calcular uma matriz auxiliar $C' \in \mathbb{R}^{n \times n}$, em que n representa o número de imagens de treinamento. Dessa forma, a diagonalização passa a ter custo computacional da ordem de $O(n^3)$, tornando viável a obtenção dos autovalores e autovetores necessários.

Na implementação, o cálculo dos autovalores e autovetores foi realizado utilizando a função **numpy.linalg.eigh**, desenvolvida para matrizes simétricas, como é o caso de C'. Essa função foi escolhida em vez de **numpy.linalg.eig**, pois aproveita a simetria da matriz para calcular autovalores reais e obter os autovetores com maior estabilidade numérica. Com isso, reduz-se a influência de erros de arredondamento que podem ocorrer durante o processo de diagonalização.

4 Resultados e Discussão

4.1 Apresentação de Dados e Desempenho Computacional

O sistema foi submetido a testes experimentais com o objetivo de avaliar a eficácia da redução de dimensionalidade proporcionada pelo método *Eigenfaces*. Observou-se que a etapa de treinamento, responsável pela centralização dos dados, cálculo da matriz de covariância e obtenção dos autovalores e autovetores, concentra a maior parte do custo computacional. Entretanto, como esse processamento é realizado apenas uma vez durante a inicialização do sistema, seu tempo de execução não compromete a utilização da aplicação em tempo real.

Nos testes de reconhecimento facial, verificou-se que a etapa de classificação apresenta baixo custo computacional. Após o treinamento, cada imagem capturada é projetada no espaço das componentes principais por meio de produtos escalares, permitindo que o reconhecimento seja realizado em poucos segundos. Também foi observado que o desempenho do sistema depende diretamente da escolha do limiar utilizado na Distância Euclidiana. Valores muito baixos tornaram o sistema excessivamente restritivo, aumentando a ocorrência de falsos negativos devido a pequenas variações de expressão ou iluminação. Por outro lado, valores muito elevados favoreceram a ocorrência de falsos positivos, reduzindo a capacidade do sistema de distinguir indivíduos diferentes.

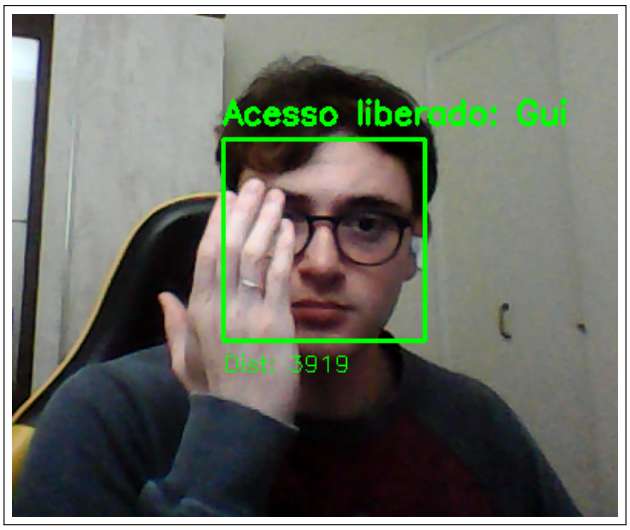


Figura 1: Limiar de aceitação de 30.000, falso positivo.



Figura 2: Limiar de aceitação de 300, falso negativo.

4.2 Aplicações Visuais

A interpretação dos resultados também foi facilitada por meio da visualização das *eigenfaces*. Para isso, os autovetores de maior importância foram convertidos novamente para a dimensão original das imagens (112×92 pixels), permitindo observar as regiões do rosto que mais contribuem para a variação entre as imagens do conjunto de treinamento.



Figura 3: Exemplos da imagem 112x92 vetorizada e reconstruída.

As imagens geradas, conhecidas como “rostos fantasmas”, evidenciam os padrões capturados pelo algoritmo e ilustram onde se concentram as principais variações presentes na base de dados. Note que, quanto mais rostos no banco de dados, mais difícil para o olho humano entender as *eigenfaces* sobrepostas.

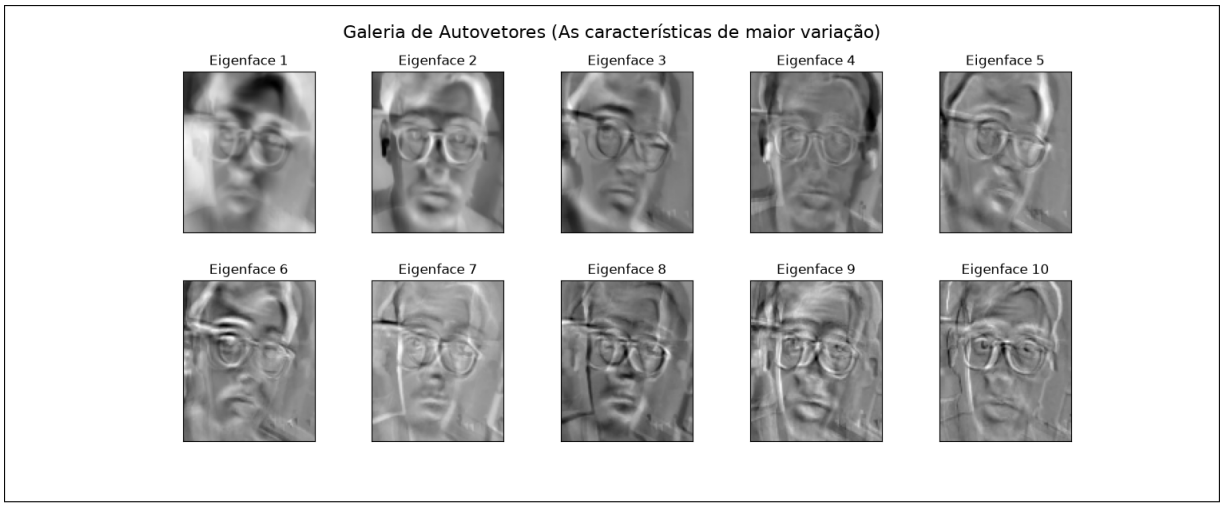


Figura 4: Autovetores gerados a partir de uma pessoa no banco de dados, utilizando óculos.

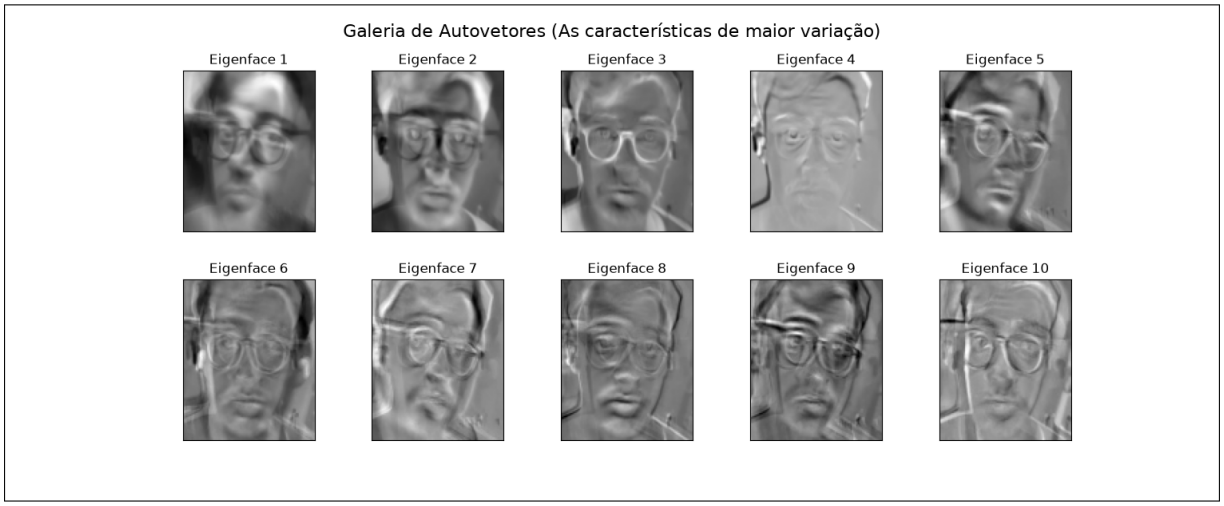


Figura 5: Autovetores gerados de uma pessoa no banco de dados, capturada com e sem óculos.

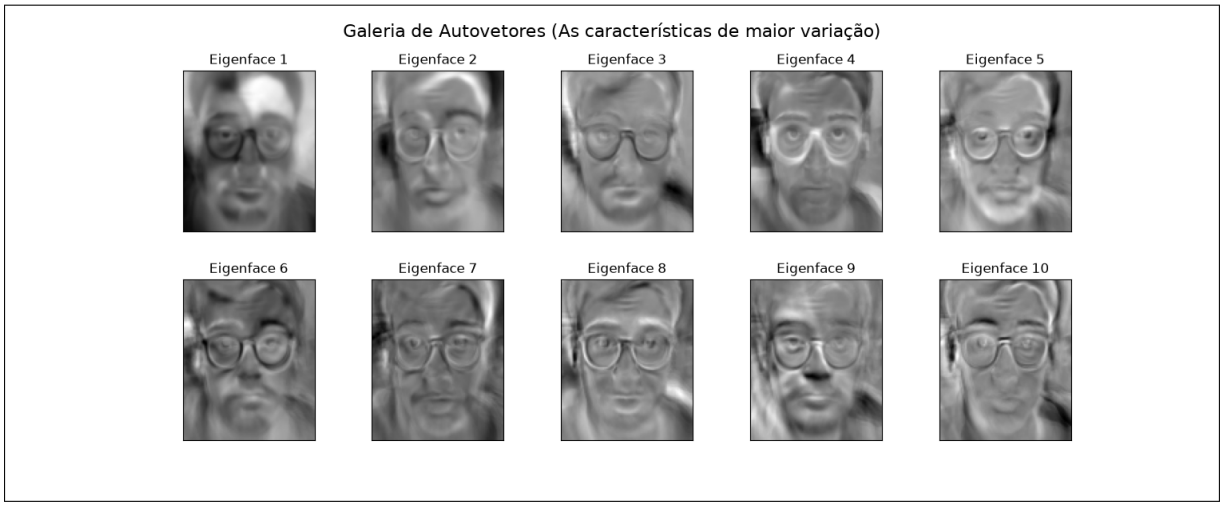


Figura 6: Representação das Eigenfaces para um banco de dados contendo duas pessoas distintas.

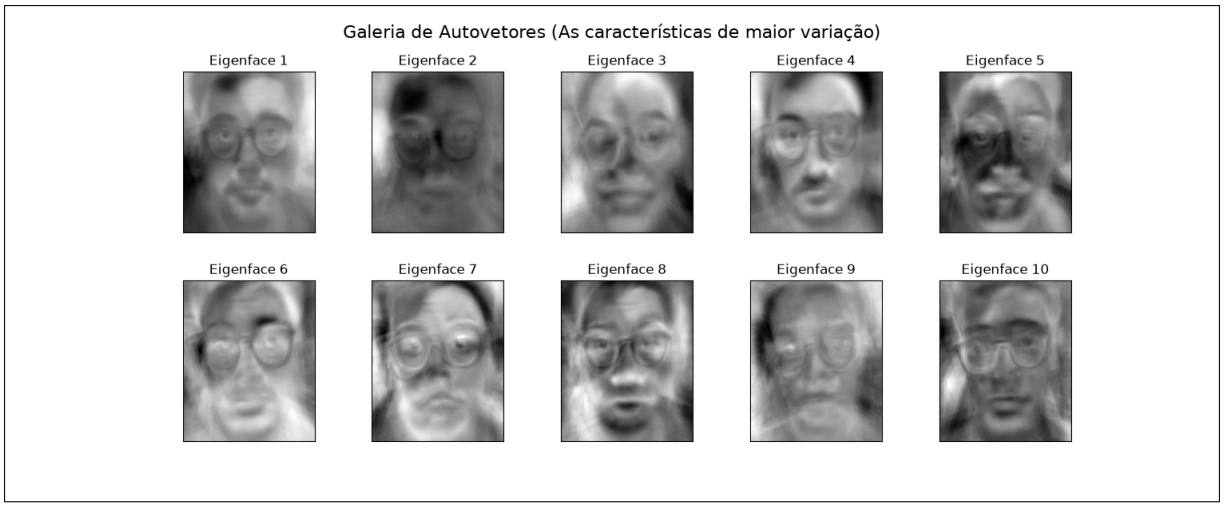


Figura 7: Representação das Eigenfaces para um banco de dados contendo quatro pessoas distintas.

4.3 Acurácia em componentes principais (k)

Além disso, o gráfico de acurácia em função do número de componentes principais (k) demonstra que a aplicação da técnica de PCA foi eficiente para o conjunto de imagens utilizado. Observa-se que o sistema não necessita utilizar todos os componentes principais para obter um bom desempenho no reconhecimento facial. Com um número reduzido de componentes, o modelo já alcança 100% de acurácia no conjunto de teste.

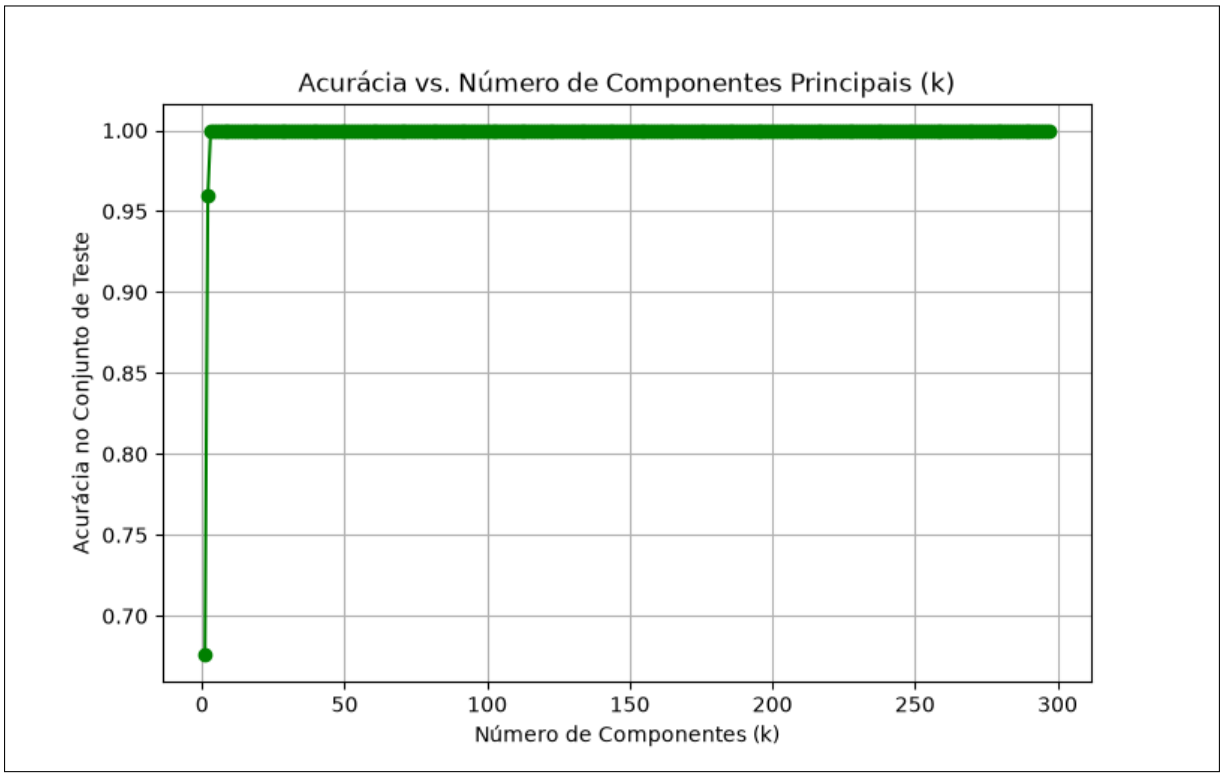


Figura 8: Gráfico de Acurácia vs. Número de Componentes Principais (k).

Esse resultado indica que as primeiras *Eigenfaces* concentram as informações mais relevantes para distinguir os indivíduos presentes na base de dados. À medida que novos componentes

são adicionados, o ganho na taxa de reconhecimento torna-se inexistente, enquanto o custo computacional continua aumentando.

Dessa forma, a análise preliminar do gráfico permite concluir que a utilização de um valor reduzido de k é suficiente para preservar a precisão no subconjunto de testes, além de diminuir a quantidade de operações matemáticas necessárias. Contudo, a aplicação do sistema na webcam em tempo real revelou um importante fenômeno de ilusão estatística atrelado a esse limite.

Apesar do gráfico apontar acurácia total com valores extremamente baixos (como $k = 3$), observou-se que esses primeiros autovetores modelaram características ambientais genéricas como diferenças na iluminação do ambiente, o fundo do cenário e o contraste da roupa e não a face em si. Esse vazamento de dados (*data leakage*) ocorreu devido à captura sequencial das imagens de treinamento em uma mesma sessão, fazendo com que o algoritmo se perdesse.

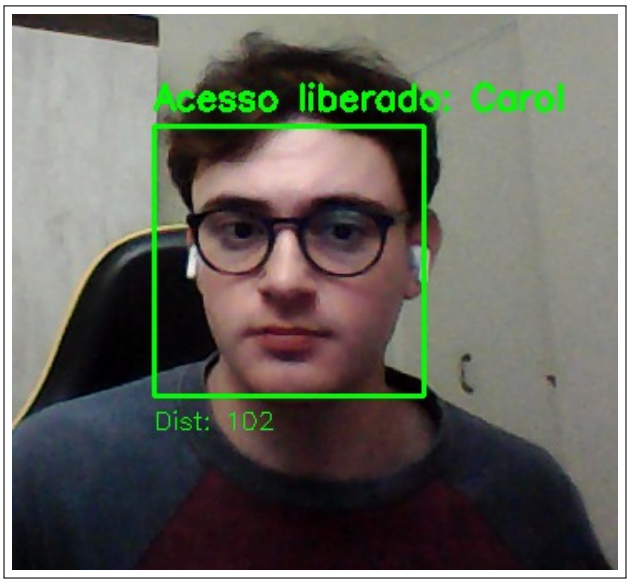


Figura 9: k=3 algoritmo perde eficiência

Para resolver esse problema e evitar falsos positivos críticos no ambiente, foi necessário sobrepor o valor ótimo matemático sugerido pelo gráfico e fixar um número de componentes mais elevado para o uso na câmera ($k \approx 60$). Esse ajuste forçou a projeção a descartar a dominância inicial da luz e do cenário, obrigando a métrica de distância a computar os traços anatômicos do usuário e restabelecendo a confiabilidade do reconhecimento facial.



Figura 10: k=60 algoritmo eficiênte

4.4 Sensibilidade a Acessórios e Variações de Pose

Durante os testes de reconhecimento em tempo real utilizando a webcam, observou-se uma limitação do sistema ao distinguir diferentes subcategorias pertencentes ao mesmo indivíduo. Conforme ilustrado nas Figuras 11, o usuário aparece sem óculos em ambas as capturas. Apesar disso, pequenas variações no ângulo ou na posição da face foram suficientes para que o sistema alternasse a classificação entre as classes *Marcos_sem_Oculos* (Distância: 2569) e *Marcos_com_Oculos* (Distância: 2605).

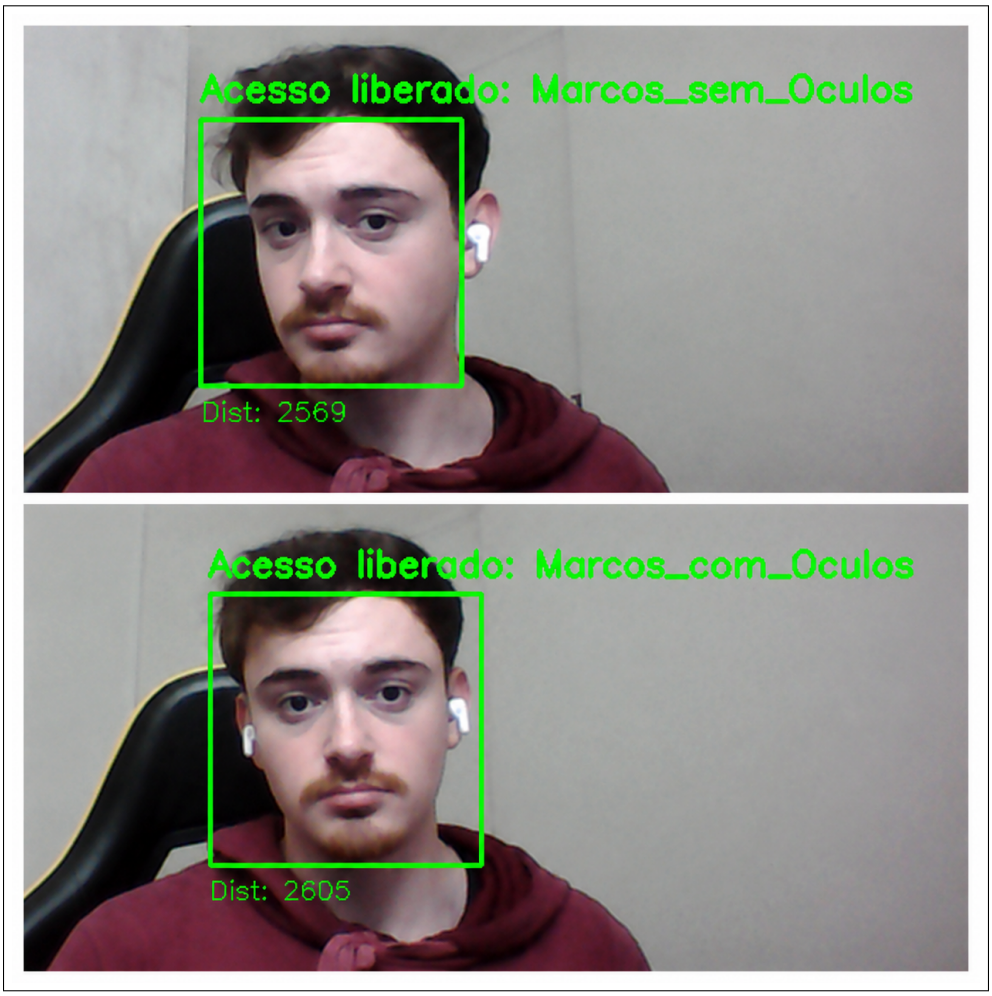


Figura 11: Oscilação na predição de classe devido à sensibilidade a pequenas variações de pose.

Esse comportamento pode ser explicado pela proximidade entre as distâncias euclidianas calculadas para as duas classes. Como ambas representam o mesmo indivíduo, suas projeções no espaço das componentes principais são muito semelhantes, fazendo com que pequenas variações de pose, iluminação ou expressão facial influenciem a decisão do classificador. Esse resultado evidencia uma limitação do método baseado em Eigenfaces, que considera apenas a menor distância no espaço holístico projetado e não incorpora informações adicionais ou filtros capazes de diferenciar características locais específicas com precisão isolada.

4.5 Interpretação Analítica

A análise das *eigenfaces* permitiu observar como o algoritmo PCA organiza as componentes principais de acordo com a variância explicada pelos respectivos autovalores ($\lambda_1 > \lambda_2 > \cdots > \lambda_n$). As primeiras componentes concentraram as informações de maior relevância presentes no conjunto de treinamento, enquanto as componentes de ordem mais elevada passaram a representar variações menos significativas.

Durante a inspeção visual das primeiras *eigenfaces*, foi possível identificar padrões relacionados à iluminação, ao formato do rosto e a outras características faciais. Em alguns casos, observou-se que atributos como a presença de óculos também influenciaram a construção das componentes principais, indicando que esse tipo de variação faz parte das informações consideradas pelo algoritmo durante o treinamento.

Nos testes realizados, verificou-se que a utilização de imagens do mesmo indivíduo em diferentes condições, incluindo registros com e sem óculos, contribuiu para um reconhecimento mais consistente. A diversidade das imagens de treinamento permitiu que o PCA representasse melhor as variações de aparência de cada pessoa, reduzindo a influência de características específicas, como acessórios, e favorecendo a extração de informações mais relacionadas aos traços faciais permanentes.

Esses resultados mostram que a representação das imagens por meio das componentes principais preservou as características mais importantes para o reconhecimento facial, ao mesmo tempo em que reduziu significativamente a dimensionalidade dos dados. Dessa forma, foi possível obter um sistema com menor custo computacional sem comprometer o desempenho observado nos testes realizados.

5 Conclusão

O presente trabalho demonstrou com sucesso a aplicação prática de conceitos avançados de Álgebra Linear Computacional no desenvolvimento de um sistema de reconhecimento facial. A utilização da técnica de Análise de Componentes Principais (PCA) comprovou ser uma abordagem altamente eficiente para o tratamento de imagens em alta resolução, contornando a maldição da dimensionalidade ao comprimir dados de $\mathbb{R}^{10304}$ para um subespaço de baixa dimensão sem perda significativa de informação discriminativa.

Os resultados obtidos evidenciam que o modelo é capaz de alcançar precisão máxima no conjunto de testes utilizando um número reduzido de componentes (k), validando a hipótese de que a maior parte da variância facial se concentra nas primeiras *Eigenfaces*. Apesar das limitações observadas quanto à sensibilidade a variações locais, como o uso de óculos e pequenas mudanças de pose, o sistema operou com extrema rapidez e confiabilidade no cálculo das projeções ortogonais e na classificação por distância euclidiana geométrica em tempo real.

Conclui-se que o domínio de decomposições matriciais e do problema espectral fornece uma base sólida não apenas para o reconhecimento biométrico, mas para diversas áreas da Ciência de Dados e Visão Computacional, cumprindo integralmente os objetivos propostos para o protótipo de controle de acesso.

Apêndice A – Implementação da Rotina Matemática (PCA)

O código-fonte Python a seguir demonstra a aplicação prática das teorias descritas na Seção 2, implementando o cálculo da média, centralização de dados, matriz de covariância e o problema espectral usando `numpy`.

```python
def treinar_eigenfaces(matriz_imagens):
    # 1. Calculo do rosto medio
    rosto_medio = np.mean(matriz_imagens, axis=0)

    # 2. Centralizacao de dados
    X_centralizado = matriz_imagens - rosto_medio

    # 3. Matriz de Covariancia (truque de baixa dimensao)
    C = np.dot(X_centralizado, X_centralizado.T)

    # 4. Solucao do Problema Espectral (Teorema Espectral)
    autovalores, autovetores_aux = np.linalg.eigh(C)

    # 5. Ordenacao dos autovalores (maior para menor variancia)
    indices_ordenados = np.argsort(autovalores)[::-1]
    autovetores_aux = autovetores_aux[:, indices_ordenados]

    # 6. Mudanca de Base para o espaco original
    eigenfaces = np.dot(X_centralizado.T, autovetores_aux).T

    # 7. Normalizacao Euclidiana dos Autovetores
    for i in range(eigenfaces.shape[0]):
        norma = np.linalg.norm(eigenfaces[i])
        if norma > 0:
            eigenfaces[i] = eigenfaces[i] / norma

    return rosto_medio, eigenfaces, autovalores
```

Listing 1: Função de Treinamento PCA do arquivo EigenFacePCA.py

Referências

STRANG, Gilbert. *Linear Algebra and Its Applications*. 4. ed. Belmont: Thomson Brooks/Cole, 2006.

LAY, David C. *Linear Algebra and Its Applications*. 5. ed. Boston: Pearson, 2015.

HARRIS, Charles R. et al. Array programming with NumPy. *Nature*, v. 585, p. 357-362, 2020.

TURK, Matthew; PENTLAND, Alex. Eigenfaces for Recognition. *Journal of Cognitive Neuroscience*, v. 3, n. 1, p. 71-86, 1991.

BAJWA, Usama Ijaz; TAJ, Imtiaz Ahmad; ANWAR, Muhammad Waqas; WANG, Xuan. A Multifaceted Independent Performance Analysis of Facial Subspace Recognition Algorithms. *PLoS One*, v. 8, n. 2, 2013. Disponível em: `https://pmc.ncbi.nlm.nih.gov/articles/PMC3581524/`. Acesso em: 6 jul. 2026.

CAO, Xue. *Eigenface-based face recognition*. YouTube, 1 maio 2013. 1 vídeo (10 min. 22 seg.). Disponível em: `https://youtu.be/e-d7WHdvBIk?si=2bQovEQ2phUgfFmF`. Acesso em: 6 jul. 2026.

HE, Henry. *eigenfaces*. GitHub, 2020. Repositório contendo o código eigenfaces.ipynb. Disponível em: https://github.com/heliumind/eigenfaces/blob/master/code/eigenfaces.ipynb. Acesso em: 6 jul. 2026.

WIKIPEDIA. *Principal component analysis*. In: Wikipedia, the free encyclopedia. Flórida: Wikimedia Foundation, 2026. Disponível em: https://en.wikipedia.org/wiki/Principal_component_analysis. Acesso em: 6 jul. 2026.

BRAGA, Luiz Filipe Zenicola. *[Substitua pelo Título Exato do Trabalho]*. [Ano de Publicação]. Trabalho Acadêmico – Universidade de São Paulo (USP), São Paulo, [Ano]. Disponível em: https://bdta.abcd.usp.br/directbitstream/46b694a0-715b-462d-a8b7-546ea4ef259d/Braga_Luiz_Filipe_Zenicola.pdf. Acesso em: 6 jul. 2026.